

```
:- [curs11mc2ID].
```

```
:- op(500,xfx,xor).
```

```
P xor Q :- P, not(Q) ; Q, not(P).
```

```
implica(P,Q) :- not(P) ; Q.
```

```
echiv(P,Q) :- implica(P,Q), implica(Q,P).
```

```
/* Cazul in care in conditie (aici, member((X,Y),[(a,1),(a,2),(b,1),(c,3),(a,1)])) exista  
variabile (aici, Y) care nu apar in termenii (aici, X) colectati in lista (aici, L):
```

```
?- setof(X, member((X,Y),[(a,1),(a,2),(b,1),(c,3),(a,1)]),L).
```

```
?- bagof(X, member((X,Y),[(a,1),(a,2),(b,1),(c,3),(a,1)]),L).
```

```
?- findall(X, member((X,Y),[(a,1),(a,2),(b,1),(c,3),(a,1)]),L).
```

```
Cuantificare existentiala pentru variabila Y care nu apare in termenul X:
```

```
?- setof(X, Y^member((X,Y),[(a,1),(a,2),(b,1),(c,3),(a,1)]),L).
```

```
?- bagof(X, Y^member((X,Y),[(a,1),(a,2),(b,1),(c,3),(a,1)]),L).
```

```
*/
```

```
/* Conectorii logici nu sunt independenti unul fata de altul. De exemplu, ce tupluri de  
valori booleene sunt valori pentru:
```

```
    (non p, p si q, p sau q, p=>q, p<=>q, p xor q),
```

```
unde p si q sunt enunturi arbitrare? Care este numarul acestor tupluri?
```

```
*/
```

```
afislista([]).
```

```
afislista([H|T]) :- write(H), nl, afislista(T).
```

```

listaBool([]).
listaBool([H|T]) :- member(H,[false,true]), listaBool(T).

tuplu(P,Q,A,B,C,D,E,F) :- echiv(A,not(P)), echiv(B,(P,Q)), echiv(C,P;Q),
                           echiv(D,implica(P,Q)), echiv(E,echiv(P,Q)), echiv(F,P xor Q).

cate :- setof((A,B,C,D,E,F), (P,Q)^(listaBool([P,Q,A,B,C,D,E,F]),
                                     tuplu(P,Q,A,B,C,D,E,F)), L), afislista(L), nl, length(L,N), write(N).

cateacestea :- setof((A,B,C), (P,Q)^(listaBool([P,Q,A,B,C]),
                                     echiv(A,not(P)), echiv(B,implica(P,Q)), echiv(C,P;Q)), L),
               afislista(L), nl, length(L,N), write(N).

exprimplic :- setof((A,B), (P,Q)^(listaBool([P,Q,A,B]),
                                     echiv(A,P;Q), echiv(B,implica(not(P),Q))), L),
               afislista(L), nl, length(L,N), write(N).

expresieimplic :- findall((A,B), (listaBool([P,Q,A,B]),
                                     echiv(A,P;Q), echiv(B,implica(not(P),Q))), L),
                  afislista(L), nl, length(L,N), write(N).

catetotal :- findall((A,B,C,D,E,F), (listaBool([P,Q,A,B,C,D,E,F]),
                                     tuplu(P,Q,A,B,C,D,E,F)), L), afislista(L), nl, length(L,N), write(N).

cateacesteatotal :- findall((A,B,C), (listaBool([P,Q,A,B,C]),
                                     echiv(A,not(P)), echiv(B,implica(P,Q)), echiv(C,P;Q)), L),
                    afislista(L), nl, length(L,N), write(N).

```

```
tabel :- setof((P,Q,A,B,C,D,E,F), (listaBool([P,Q,A,B,C,D,E,F]),
    tuplu(P,Q,A,B,C,D,E,F)), L), afislista(L), nl, length(L,N), write(N).
```

```
tabeltotal :- findall((P,Q,A,B,C,D,E,F), (listaBool([P,Q,A,B,C,D,E,F]),
    tuplu(P,Q,A,B,C,D,E,F)), L), afislista(L), nl, length(L,N), write(N).
```

```
/*
```

```
?- bagof((A,B,C,D,E,F), (listaBool([P,Q,A,B,C,D,E,F]), tuplu(P,Q,A,B,C,D,E,F)), L),
write(L).
```

```
?- setof((A,B), (listaBool([P,Q,A,B]), echiv(A,P;Q), echiv(B,implica(not(P),Q))), L),
afislista(L).
```

```
?- bagof((A,B), (listaBool([P,Q,A,B]), echiv(A,P;Q), echiv(B,implica(not(P),Q))), L),
afislista(L).
```

```
*/
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
/* Fie A,B,C multimi si x un element arbitrar, fixat.
```

```
Urmatoarele variabile Prolog (booleene) vor reprezenta enunturile:
```

```
_a: x apartine lui A
```

```
_b: x apartine lui B
```

```
_c: x apartine lui C
```

```
Atunci urmatoarele expresii booleene vor reprezenta aceste enunturi:
```

```
_a;_b <=> x apartine lui AUB
```

```
_a,_b <=> x apartine lui A^B (^ = intersectie)
```

```
_a,not(_b) <=> x apartine lui A\B
```

```
_a,not(_b) ; _b,not(_a) <=> x apartine lui A/\B (/ \ = diferenta simetrica),
```

```

deci: _a xor _b <=> x apartine lui A/\B
reunx(_a,_b) = true <=> x apartine lui A\B
intersx(_a,_b) = true <=> x apartine lui A^B
difx(_a,_b) = true <=> x apartine lui A\B
difsimx(_a,_b) = true <=> x apartine lui A/\B
Egalitatea A=B inseamna ca, pentru orice x, are loc:
    x apartine lui A <=> x apartine lui B,
adica: echiv(_a,_b).
Incluziunea A<=B (A inclusa nestrict in B) inseamna ca, pentru orice x, are loc:
    x apartine lui A => x apartine lui B,
adica: implica(_a,_b).
Notam cu  $\emptyset$  multimea vida. Aceasta este multimea fara elemente, adica, pentru orice x,
proprietatea:
    x apartine  $\emptyset$ 
este falsa.
Daca M si T sunt multimi cu  $M \subseteq T$ , iar x apartine lui T, atunci, notand cu:
-M =  $T \setminus M$ , avem ca proprietatea:
    x apartine -M <=> x apartine  $T \setminus M$  <=> x apartine T si non(x apartine M)
    <=> true si non(x apartine M) <=> non(x apartine M)
*/

```

```

reunx(_a,_b) :- _a ; _b.

```

```

intersx(_a,_b) :- _a , _b.

```

```

difx(_a,_b) :- _a , not(_b).

```

```

difsimx(_a,_b) :- _a xor _b.

inclstrictx(_a,_b) :- implica(_a,_b), not(echiv(_a,_b)).

listaValBool(L) :- listaBool(L), write(L), nl.

/* Cu notatiile de mai sus, distributivitatea reuniunii fata de intersectie:
   
$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

se demonstreaza prin distributivitatea disjunctiei fata de conjunctie:
pentru x-ul arbitrar, demonstram ca:
   x apartine lui  $A \cup (B \cap C) \iff x$  apartine lui  $(A \cup B) \cap (A \cup C)$ ,
adica:
    $_a \text{ sau } (_b \text{ si } _c) \iff (_a \text{ sau } _b) \text{ si } (_a \text{ sau } _c)$ ,
sau, cu predicatele de mai sus:
   echiv(reunx(_a,intersx(_b,_c)),intersx(reunx(_a,_b),reunx(_a,_c))).
*/

distribreunfdinters(_a,_b,_c) :-
    echiv(reunx(_a,intersx(_b,_c)),intersx(reunx(_a,_b),reunx(_a,_c))).

demdistribreunfdinters :- not((listaValBool([_a,_b,_c]),
    not(distribreunfdinters(_a,_b,_c)))).

/* Analog, sa demonstram distributivitatea intersectiei fata de reuniune:
   
$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

*/

```

```

distribintersfdreun(_a,_b,_c) :-
    echiv(intersx(_a,reunx(_b,_c)), reunx(intersx(_a,_b),intersx(_a,_c))).

demdistribintersfdreun :- not((listaValBool([_a,_b,_c]),
    not(distribintersfdreun(_a,_b,_c)))).

% Asociativitatea diferentei simetrice:  $A \setminus (B \setminus C) = (A \setminus B) \setminus C$ :

asocdifsim(_a,_b,_c) :-
    echiv(difsimx(_a,difsimx(_b,_c)), difsimx(difsimx(_a,_b),_c)).

demasocdifsim :- not((listaValBool([_a,_b,_c]), not(asocdifsim(_a,_b,_c)))).

/* Sa demonstram ca, daca multimea A este nevida, atunci  $\emptyset < A$  (< este incluziunea stricta),
adica, notand cu  $\neq$  nonegalitatea:  $A \neq \emptyset \Rightarrow \emptyset < A$ .
*/

vidainclstrictnevida(_a) :-
    implica(not(echiv(_a,false)), inclstrictx(false,_a)).

vidainclstrictnevida :- not((listaValBool([_a]),
    not(vidainclstrictnevida(_a)))).

/* Reuniunea (la dreapta) pastreaza incluziunile nestricte:
 $A \leq B \Rightarrow A \cup C \leq B \cup C$ .
*/

```

```
reunpastrincl(_a,_b,_c) :- implica(implica(_a,_b),
                                   implica(reunx(_a,_c),reunx(_b,_c))).
```

```
demreunpastrincl :- not((listaValBool([_a,_b,_c]),
                           not(reunpastrincl(_a,_b,_c)))).
```

```
/* Demonstram ca, daca  $A \leq T$  si  $B \leq T$ , iar x apartine lui T, atunci:
( $A \cup B = T$  si  $A \cap B = \emptyset$ )  $\Leftrightarrow A = -B \Leftrightarrow B = -A$ .
    Amintesc ca:
not(_a)  $\Leftrightarrow$  x apartine lui -A.
*/
```

```
particomplem(_a,_b) :- echiv(reunx(_a,_b),true), echiv(intersx(_a,_b),false).
```

```
complemlui(_a,_b) :- echiv(_a,not(_b)).
```

```
charparticomplem(_a,_b) :- echiv(particomplem(_a,_b), complemlui(_a,_b)),
                           echiv(complemlui(_a,_b), complemlui(_b,_a)).
```

```
demcharparticomplem :- not((listaValBool([_a,_b]),
                              not(charparticomplem(_a,_b)))).
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
/* Acum sa calculam rezultatele operatiilor cu multimi si sa determinam daca au loc
relatiile intre multimi pentru multimi finite date prin liste in Prolog:
```

*/

reun(A,B,AUB) :- append(A,B,C), elimdupl(C,AUB).

reuniune(A,B,AUB) :- setof(X, (member(X,A) ; member(X,B)), AUB), !.
reuniune(_,_,[]).

intersmult(A,B,AiB) :- inters(A,B,I), elimdupl(I,AiB).

inters([],_,[]).
inters([H|T],M,[H|L]) :- member(H,M), !, inters(T,M,L).
inters([_|T],M,L) :- inters(T,M,L).

intersectie(A,B,AiB) :- setof(X, (member(X,A) , member(X,B)), AiB), !.
intersectie(_,_,[]).

difmult(A,B,AminusB) :- dif(A,B,D), elimdupl(D,AminusB).

dif([],_,[]).
dif([H|T],M,L) :- member(H,M), !, dif(T,M,L).
dif([H|T],M,[H|L]) :- dif(T,M,L).

difermult(A,B,AminusB) :- difer(A,B,D), elimdupl(D,AminusB).

difer(M,[],M).
difer(M,[H|T],D) :- sterge(H,M,L), difer(L,T,D).

```
diferenta(A,B,AminusB) :- setof(X, (member(X,A),not(member(X,B))), AminusB), !.  
diferenta(_,_,[]).
```

```
difsim(A,B,ADB) :- dif(A,B,AminusB), dif(B,A,BminusA),  
                  reun(AminusB,BminusA,ADB).
```

```
diferentasimetrica(A,B,ADB) :- setof(X, (member(X,A) xor member(X,B)), ADB), !.  
diferentasimetrica(_,_,[]).
```

```
% Generarea sublistelor/submultimilor unei liste/multimi:
```

```
sublista([],_).  
sublista([H|T],[H|L]) :- sublista(T,L).  
sublista([H|T],[_|L]) :- sublista([H|T],L).
```

```
sublistele(L,LS) :- setof(S, sublista(S,L), LS).
```

```
% Testarea incluziunii nestricte intre multimi:
```

```
inclusa([],_).  
inclusa([H|T],M) :- member(H,M), inclusa(T,M).
```

```
/* Testarea egalitatii a doua liste Prolog ca multimi, adica proprietatea ca au aceleasi  
elemente: */
```

```
egalmult(A,B) :- inclusa(A,B), inclusa(B,A).
```

% Testarea incluziunii stricte intre multimi:

inclusastrict(A,B) :- inclusa(A,B), not(egalmult(A,B)).

%%%

/* Pentru ultimul exercitiu din prima parte a Seminarului I, putem verifica faptul ca, pentru o anumita multime T (finita, pentru ca va fi data printr-o lista in Prolog), avem proprietatea:

(oricare ar fi $A \subseteq T$ si $B \subseteq T$)[daca
 $f: P(T) \rightarrow P(A) \times P(B)$, oricare ar fi $X \subseteq T$, $f(X) = (X^A, X^B) \Rightarrow$
 ((f e injectiva $\Leftrightarrow A \cup B = T$) si (f e surjectiva $\Leftrightarrow A \cap B = \emptyset$))].

*/

verifenuent(T) :- not((sublista(A,T), sublista(B,T),
 write(A), tab(1), write(B), nl,
 not((charinjF(A,B,T), charsurjF(A,B,T))))).

charinjF(A,B,T) :- echiv(injF(A,B,T), reunABT(A,B,T)).

reunABT(A,B,T) :- reun(A,B,R), egalmult(R,T).

injF(A,B,T) :- not((sublista(X,T), sublista(Y,T), not(egalmult(X,Y)),
 inters(X,A,XiA), inters(Y,A,YiA), egalmult(XiA,YiA),
 inters(X,B,XiB), inters(Y,B,YiB), egalmult(XiB,YiB))).

charsurjF(A,B,T) :- echiv(surjF(A,B,T), intersAB0(A,B)).

```
intersAB0(A,B) :- inters(A,B,I), I=[].
```

```
surjF(A,B,T) :- not((sublista(X,A), sublista(Y,B), not((sublista(Z,T),  
inters(Z,A,ZiA), egalmult(ZiA,X),  
inters(Z,B,ZiB), egalmult(ZiB,Y)))))).
```

```
/* Verificare pentru orice multime  $T=\{x_1,x_2,\dots,x_K\}$  de cardinal  $K\leq N$ :  
verifenuntul(N) -> cu multimea T de cardinal K reprezentata de lista de indici  
ai elementelor sale: [K,...,2,1];  
verifenuntulx(N) -> cu multimea T de cardinal K reprezentata de lista  
de constante Prolog doua cate doua distincte [xK,...,x2,x1]:  
*/
```

```
calefis('d:/tempwork/').  
numefis('verifptTdecard').  
extfis('.txt').
```

```
listaT(0,[]).  
listaT(K,[K|T]) :- K>0, PK is K-1, listaT(PK,T).
```

```
verifenuntul(N) :- (N>0, !, PN is N-1, verifenuntul(PN) ; true),  
listaT(N,T), write('T='), write(T), nl,  
calefis(Cale), numefis(NumeFis), extfis(Extensie),  
atom_concat(NumeFis,N,NumeFisier),  
atom_concat(NumeFisier,Extensie,NumeFisierExtensie),  
atom_concat(Cale,NumeFisierExtensie,CaleNumeFisierExtensie),
```

```

        tell(CaleNumeFisierExtensie), verifenu(T), told.

listaTx(0,[]).
listaTx(K,[XK|T]) :- K>0, atom_concat(x,K,XK), PK is K-1, listaTx(PK,T).

verifenuTx(N) :- (N>0, !, PN is N-1, verifenuTx(PN) ; true),
    listaTx(N,T), write('T='), write(T), nl,
    calefis(Cale), numefis(NumeFis), extfis(Extensie),
    atom_concat(NumeFis,N,NumeFisier),
    atom_concat(NumeFisier,Extensie,NumeFisierExtensie),
    atom_concat(Cale,NumeFisierExtensie,CaleNumeFisierExtensie),
    tell(CaleNumeFisierExtensie), verifenu(T), told.

% Mai avantajos, evitand calculele repetitive:

verifenuTx(N) :- calefis(Cale), numefis(NumeFis),
    extfis(Extensie), auxverifenuTx(N,_,Cale,NumeFis,Extensie).

auxverifenuTx(N,T,Cale,NumeFis,Extensie) :-
    (N>0, !, PN is N-1, auxverifenuTx(PN,L,Cale,NumeFis,Extensie),
    atom_concat(x,N,XN), T=[XN|L] ; T=[]),
    write('T='), write(T), nl,
    atom_concat(NumeFis,N,NumeFisier),
    atom_concat(NumeFisier,Extensie,NumeFisierExtensie),
    atom_concat(Cale,NumeFisierExtensie,CaleNumeFisierExtensie),
    tell(CaleNumeFisierExtensie), verifenu(T), told.

```

```
/* Diferenta intre timpii de executie e, totusi, minora:
?- Init is cputime, verifentulx(5), Fin is cputime, Dif is Fin-Init, write(Dif), write('
secunde').
?- Init is cputime, verifentx(5), Fin is cputime, Dif is Fin-Init, write(Dif), write('
secunde').
*/
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% Relatie binara de la A la B:
```

```
relbin(R,A,B) :- prodcartmult(A,B,P), sublista(R,P).
```

```
/* Functionalitatea unei relatii binare R, i.e. faptul de a fi functie partiala, pentru
codomeniul lui R dat de o lista de constante, pentru ca nonunificarea sa insemne
nonegalitate: */
```

```
functional(R) :- not((member((X,Y),R), member((X,Z),R), Y\=Z)).
```

```
% Totalitatea unei relatii binare R cu domeniul A:
```

```
totala(R,A) :- not((member(X,A), not(member((X,_),R)))).
```

```
% Functiile partiale totale sunt chiar functiile:
```

```
fctparttot(R,A) :- functional(R), totala(R,A).
```

% Determinarea functiilor de la A la B se poate face astfel:

efunctie(R,A) :- functionala(R), totala(R,A).

ofunctie(R,A,B) :- relbin(R,A,B), efunctie(R,A).

functii(A,B,LF) :- setof(F, ofunctie(F,A,B), LF), !.
functii(_,_,[]).

% Determinarea functiilor de la A la B, mai avantajos: functie(F,A,B), F:A->B

functie([],[],_).
functie([(H,FH)|L],[H|T],B) :- member(FH,B), functie(L,T,B).

functiile(A,B,LF) :- setof(F, functie(F,A,B), LF), !.
functiile(_,_,[]).

% Inversa unei relatii binare:

invrel(R,I) :- setof((Y,X), member((X,Y),R), I), !.
invrel(_,[]).

/* Injectivitatea unei relatii binare R, pentru domeniul lui R dat de o lista de constante,
pentru ca nonunificarea sa insemne nonegalitate: */

injectiva(R) :- not((member((X,Y),R), member((U,Y),R), X\=U)).

```

inj(R) :- invrel(R,I), functionala(I).

% Surjectivitatea unei relatii binare R cu codomeniul B:

surjectiva(R,B) :- not((member(Y,B), not(member( (_,Y),R)))).

surj(R,B) :- invrel(R,I), totala(I,B).

/* Sa demonstrem, pentru multimi A si B date, ca functiile de la A la B a caror inversa e
tot functie sunt exact functiile bijective: */

% cu afisarea tuturor functiilor de la A la B:

inversafct(A,B) :- not((functie(F,A,B), write(F), nl, invrel(F,I),
    not(echiv(efunctie(I,B), (injectiva(F), surjectiva(F,B)))))).

% recursiv, fara afisare:

inversaetotfct(A,B) :- functiile(A,B,LF), auxinvfct(B,LF).

auxinvfct(_,[]).
auxinvfct(B,[F|LF]) :- invrel(F,I),
    echiv(efunctie(I,B), (injectiva(F), surjectiva(F,B))),
    auxinvfct(B,LF).

% recursiv, cu afisarea functiilor bijective de la A la B:

```

```
inversatotfct(A,B) :- functiile(A,B,LF), auxifct(B,LF).
```

```
auxifct(_,[]).
```

```
auxifct(B,[F|LF]) :- invrel(F,I),  
    (efunctie(I,B), !, write(F), nl, injectiva(F), surjectiva(F,B) ;  
    not((injectiva(F), surjectiva(F,B)))),  
    auxifct(B,LF).
```

```
/* Pentru o relatie (in particular functie) R de la A la B,  $M \subseteq A$  si  $N \subseteq B$ ,  
imaginea lui X prin R, respectiv preimaginea lui Y prin R: */
```

```
im(R,M,Im) :- setof(Y, X^(member(X,M), member((X,Y),R)), Im), !.  
im(_,_,[]).
```

```
preim(R,N,Preim) :- setof(X, Y^(member(Y,N), member((X,Y),R)), Preim), !.  
preim(_,_,[]).
```

```
surject(R,A,B) :- im(R,A,Im), egalmult(B,Im).
```

```
/* Interogati:
```

```
?- setof(F, (functie(F,[a,b,c],[1,2]), surject(F,[a,b,c],[1,2])), L), afislista(L),  
length(L,N).
```

```
?- setof(F, (functie(F,[a,b,c],[1,2]), surjectiva(F,[1,2])), L), afislista(L), length(L,N).  
*/
```

```
% Compunerea a doua relatii binare:
```

```
comp(R,S,SoR) :- setof((X,Z), Y^(member((X,Y),R), member((Y,Z),S)), SoR), !.  
comp(_,_,[]).
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% Relatie binara pe A:
```

```
relbinara(R,A) :- relbin(R,A,A).
```